

通信原理 实验报告

学号：2234112313

班级：信息 2301

姓名：杜斯博

2 信道编解码

一 实验内容 (10 分)

1.1 信道编解码的实现和验证

本实验在已有 16QAM 通信 demo 的基础上，完成信道编码与信道解码模块的设计、嵌入和验证。发送端在 CRC 校验、加扰码之后，16QAM 星座映射之前加入 Hamming(7,4) 线性分组码编码；接收端在 16QAM 解调之后、解扰之前加入对应的 Hamming(7,4) 译码和单比特纠错。

二 实验原理 (50 分)

2.1 差错控制编码的分类 (10 分)

差错控制编码可以从不同角度分类。

按处理对象划分，可分为分组码和卷积码。分组码将信息序列按固定长度分组，每组独立编码，例如 Hamming 码、BCH 码、CRC 码和 LDPC 码。卷积码的输出不仅与当前输入有关，还与前若干时刻输入有关，具有记忆特性，例如传统卷积码和 Turbo 码。

按码字集合性质划分，可分为线性码和非线性码。线性码的任意两个码字按模 2 相加后仍为合法码字，其码字集合构成二进制向量空间中的子空间。线性分组码可以用生成矩阵和校验矩阵描述，分析和实现较为方便。非线性码不满足

上述线性封闭性质。

按编码结构划分，还可以分为循环码、系统码和非系统码。循环码具有循环移位封闭性，BCH 码和 CRC 码属于典型循环码。系统码中原始信息位直接出现在码字的固定位置，非系统码则不一定保留原信息位的直接位置。

2.2 线性分组码的编码、校验、纠错原理（20 分）

设一个二进制线性分组码为 (n,k) 码，其中 k 为信息位长度， n 为编码后码字长度， $n-k$ 为监督位长度。每 k 位信息比特通过编码器生成 n 位码字。编码效率为： $R = \frac{k}{n}$ 。本实验采用 Hamming(7,4) 码，即每 4 位信息位编码成 7 位码字，编码效率为 4/7。

线性分组码的编码可以用生成矩阵 G 表示。

设输入信息向量为： $u = [u_1 u_2 u_3 u_4]$ ，编码后码字为： $v = uG \bmod 2$ 。本实验使用的 Hamming(7,4) 生成矩阵为：

$$G = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}$$

校验矩阵 H 用于判断接收码字是否满足合法码字关系。合法码字 v 应满足：

$vH^T = 0$ 。本实验使用的校验矩阵为：

$$H = \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

设接收端收到的码字为 r ，发送端正确码字为 v ，错误向量为 e ，则有关系式 $r = v + e$ ，接收端计算伴随式： $S = (v + e)H^T = vH^T + eH^T = eH^T$ 。由于合法码字满足 $vH^T=0$ ，因此伴随式 S 只与错误位置有关，而与原始发送码

字无关。对于单比特错误，S 等于 H 矩阵对应错误位置的列向量。因此接收端可以将 S 与 H 的各列进行匹配，确定错误位位置，然后将该位取反，实现单比特纠错。

2.3 码间串扰产生的原因（10 分）

实际信道和射频前端均具有有限带宽。理想脉冲信号在频域上需要无限带宽，但实际系统无法传输无限带宽信号，因此发送信号经过带限信道后，其时域波形会产生拖尾。若一个码元的拖尾延伸到其他码元的判决时刻，就会影响相邻码元判决，形成码间串扰。

2.4 成型滤波、匹配滤波的原理和作用（10 分）

成型滤波用于限制发送信号带宽，并控制码元波形的时域拖尾，使其满足无码间串扰条件。理想无码间串扰的时域条件为：
$$\begin{aligned} h(kT) &= 1, k = 0 \\ h(kT) &= 0, k \neq 0 \end{aligned}$$
，即在当前码元判决时刻，当前码元响应最大；在其他码元判决时刻，该码元响应为零。实际系统常使用升余弦滤波器或根升余弦滤波器实现近似无码间串扰传输。

三 实验结果图示及分析（30 分）

3.1 信道编解码的具体实现（20 分）

本实验基于 16QAM demo 进行修改。代码解析和注释详见末尾附注。

发送端在 qam16_tx_func.m 中完成信道编码嵌入。具体位置位于扰码之后、16QAM 调制之前：

```
[sym_bits,padBits]=hamming74_encode(sym_bits);  
  
save('qam16_hamming74_frame_info.mat','padBits');
```

这样处理的原因是信道编码的对象是二进制比特流，而 16QAM 调制的对象

是编码后的比特序列。若将编码放在调制之后, 则处理对象已变成复数星座符号, 不符合 Hamming 码的二进制线性分组码定义。

接收端在 qam16_rx_func.m 中完成信道译码嵌入。接收端首先读取发送端保存的补零数量, 随后根据信道编码后数据长度计算接收帧长度。

```
dataSymbolLen=(2048+padBits)/4*7/4;  
  
lenFrame=length(local_sync)+dataSymbolLen/64*(64+8);  
  
[rx_frame,cor_abs,th_max,index_s]=rx_package_search(rx_sig_down,local_sync,lenFrame);
```

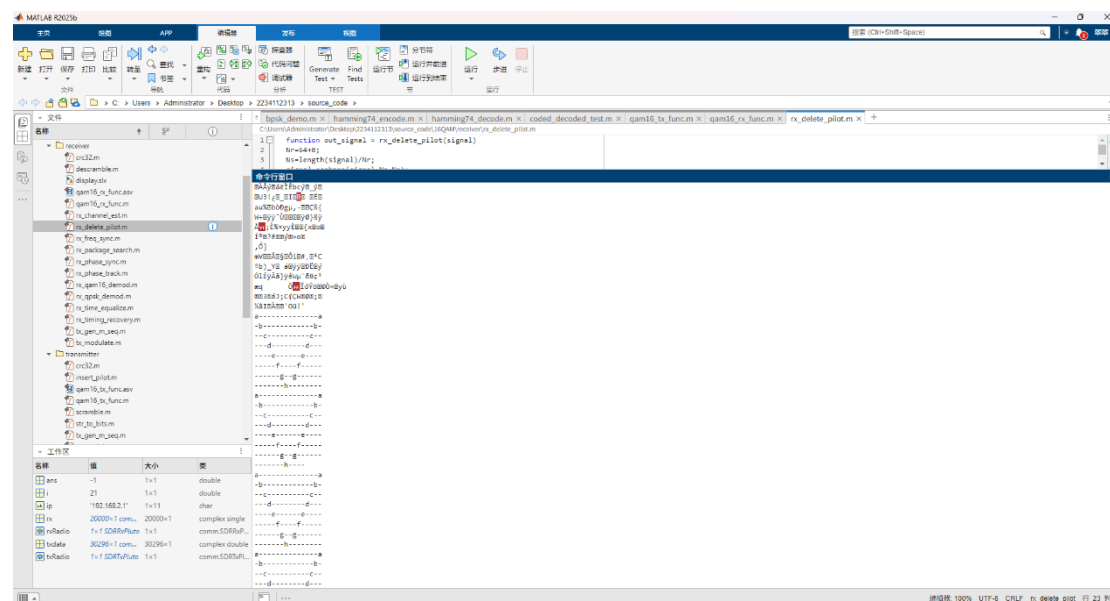
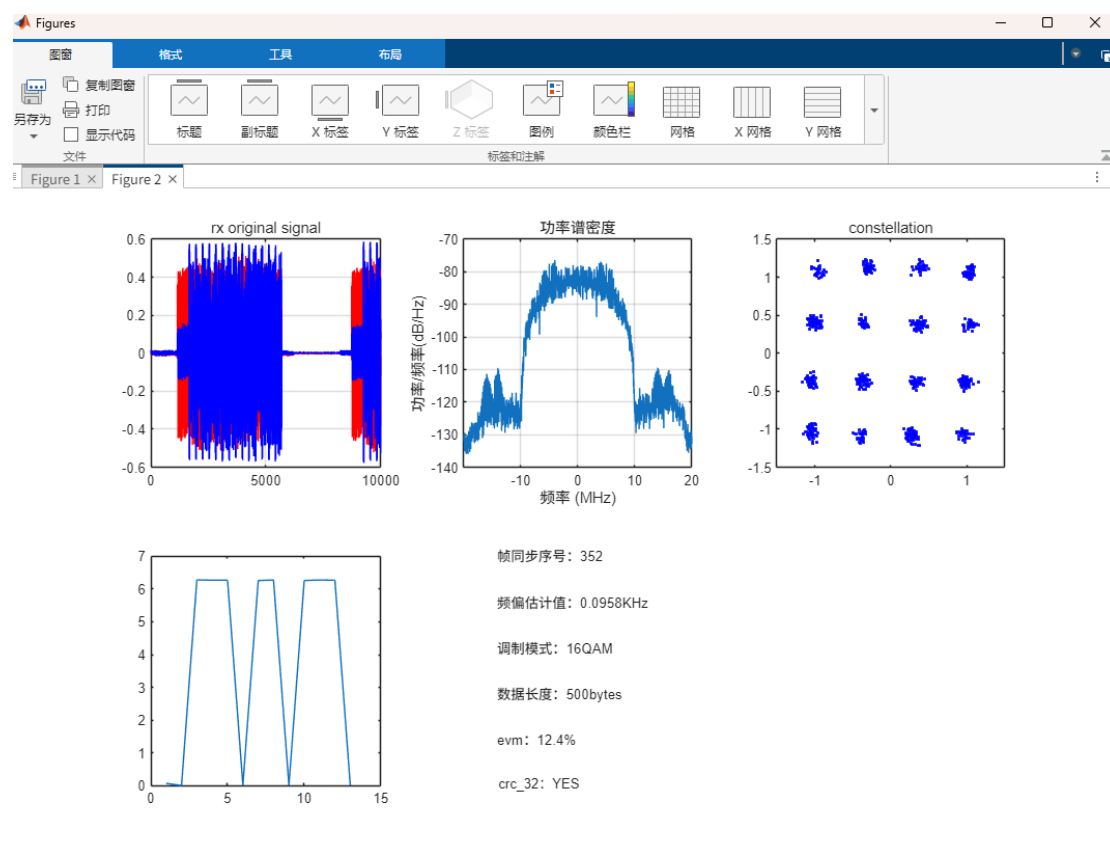
接收端在 16QAM 解调后调用译码函数:

```
[soft_bits_out,evm] = rx_qam16_demod(out_signal8);  
  
[soft_bits_out,decStats]=hamming74_decode(soft_bits_out,padBits);
```

这一步将硬判决得到的编码比特流按 7 位分组, 利用伴随式纠正单比特错误, 并恢复扰码后的原始信息比特流。随后再进行解扰和 CRC 校验。

去导频函数 rx_delete_pilot.m 将原先固定输出长度的写法修改为自动展开。

实际运行结果中, 接收端成功恢复发送端字符串, 命令行窗口显示恢复出的字符矩阵; 接收端图窗中星座图呈现 16 个聚类点, CRC 校验结果为 YES, 说明加入信道编解码后的 16QAM 链路能够正确恢复发送信息。



字符串一致，且 CRC 校验结果为 YES，说明本次采集条件下 CRC 校验通过。若以恢复出的有效比特和发送端参考比特比较，本次实验可认为有效数据恢复正常。

从接收端星座图看，16QAM 星座点形成 16 个明显聚类，说明接收端同步、均衡、解调和译码流程整体有效。频谱图显示接收信号主要能量集中在基带有效带宽内，带外能量较低，说明成型滤波和匹配滤波工作正常。接收端显示 EVM 约为 12.4%，说明星座点仍存在噪声和残余失真，但未导致本次 CRC 校验失败。

若进行未编码对照实验，可关闭发送端 Hamming 编码和接收端 Hamming 译码，保持其他参数一致，分别记录未编码和编码后的 BER。理论上，在信道错误以单比特错误为主且错误密度不太高时，Hamming(7,4) 编码后的 BER 应低于未编码系统；若信道条件很差，一个码字中经常出现多位错误，则 Hamming(7,4) 可能无法正确纠错，纠错增益会下降。

四 思考题（10 分）

4.1 调研 4G 和 5G 中的信道编码的名称、分类、码长、编码效率。

4G 中常用的信道编码包括 Turbo 码和咬尾卷积码等。Turbo 码主要用于数据信道，属于并行级联卷积码，具有接近 Shannon 极限的纠错性能。LTE Turbo 码通过速率匹配实现不同编码效率。咬尾卷积码主要用于部分控制信道，避免传统卷积码尾比特带来的额外开销。

5G 中主要采用 LDPC 码和 Polar 码。LDPC 码用于数据信道，属于线性分组码，其校验矩阵具有稀疏结构，适合高吞吐并行译码。5G 定义了两类 LDPC 基图，分别适用于不同码长和码率场景，并通过不同速率匹配方式适应不同传输

块大小和编码效率。Polar 码用于控制信道，属于基于信道极化理论构造的线性分组码，适合中短码长控制信息传输。Polar 码可通过选择可靠子信道承载信息位，不可靠子信道承载冻结位来完成编码。

4.2 选择 4.1 中的 1 种，概述编码过程和当前主流的译码方法。

以 5G 中的 LDPC 码为例。LDPC 码是一类由稀疏校验矩阵 H 定义的线性分组码。合法码字 c 满足： $Hc^T = 0$ 。编码时，发送端首先根据传输块大小选择合适的基图，然后进行码块分割、添加 CRC、基图提升和 LDPC 编码，生成满足校验方程的码字。随后经过速率匹配，得到适合当前信道条件和资源分配的编码比特流。

LDPC 主流译码方法是置信传播算法，也称 BP 算法。该算法在 Tanner 图上进行迭代消息传递，变量节点和校验节点不断交换软信息，逐步逼近最可能的发送码字。实际工程中，为降低复杂度，常使用 Min-Sum 算法及其改进形式，如归一化 Min-Sum 和偏移 Min-Sum。这些算法通过近似 BP 中复杂的双曲函数运算，降低硬件实现复杂度，同时保持较好的纠错性能。

五、代码附注

Hamming(7,4) 编码函数

```
function [codedBits, padBits] = hamming74_encode(infoBits)
% codedBits 为编码好的比特流
% padBits 为补零数，用于后续解码

infoBits = double(infoBits(:).');
% 统一整理成 double 类型的行向量

% if any(infoBits ~= 0 & infoBits ~= 1)
%     error('编码比特流必须是0和1');
```

```

% end

k = 4;
padBits = mod(k - mod(numel(infoBits), k), k);
% 补零数, 内层4-总比特数模4为补零数, 外层模4处理刚好整除

if padBits > 0
    infoBits = [infoBits zeros(1, padBits)]; % 补零
end

G = [ ...
    1 1 1 0 0 0 0; ...
    1 0 0 1 1 0 0; ...
    0 1 0 1 0 1 0; ...
    1 1 0 1 0 0 1];
% 生成矩阵用于编码

msg = reshape(infoBits, k, []).';
% 把比特流先每4分列再转置得到4*k矩阵

codewords = mod(msg * G, 2);
% 乘以生成矩阵模2得到7*n矩阵

codedBits = reshape(codewords.', 1, []);
% 把7*n矩阵先转置再合并为一行比特流
end

```

Hamming(7,4) 解码函数

```

function [infoBits, stats] = hamming74_decode(rxBits, padBits)
% rxBits 为待解码的比特流, padBits 为编码时补零数
% infoBits 为输出的已解码比特流, stats 为译码纠错信息

rxBits = double(rxBits(:).');
% 统一转为double型行向量

% if any(rxBits ~= 0 & rxBits ~= 1)
%     error('解码比特流必须是0和1');
% end
% if mod(numel(rxBits), 7) ~= 0
%     error('解码比特流的长度必须是7的倍数');
% end

```



```

G = [ ...
      1 1 1 0 0 0 0; ...
      1 0 0 1 1 0 0; ...
      0 1 0 1 0 1 0; ...
      1 1 0 1 0 0 1];
% 生成矩阵用于生成码本的16个合法码字

H = [ ...
      1 0 1 0 1 0 1; ...
      0 1 1 0 0 1 1; ...
      0 0 0 1 1 1 1];
% 校验矩阵用于校验纠错

messages = dec2bin(0:15, 4) - '0';
% 生成所有4位二进制并通过-'0'由字符转为数字比特

codebook = mod(messages * G, 2);
% 生成16个合法码字的码本, 用于纠错后反查码字

codewords = reshape(rxBits, 7, []).';
% 把接受的待解码比特流7位分组为7*n矩阵

decoded = zeros(size(codewords, 1), 4); % 纠错计数
syndromeTable = H.'; % 转置后的校验矩阵

for ii = 1:size(codewords, 1) % 开始逐个码字译码
    word = codewords(ii, :); % 取每行码字
    syndrome = mod(word * H.', 2); % 计算伴随式

    if any(syndrome) % 伴随式全零时无误, 非零时纠错
        errPos = find(all(syndromeTable == syndrome, 2), 1);
        % 计算错误的比特位数errPos
        % all(syndromeTable == syndrome, 2) 对校验矩阵逐行比较是否与伴随式
        相同
        % find 寻找到匹配的行号
        if ~isempty(errPos) % 如果有错误比特
            word(errPos) = 1 - word(errPos); % 取反
            correctedCount = correctedCount + 1;
        end
    end
end

```

```
% 纠错后根据七位码字到码本中找对应的原码
idx = find(all(codebook == word, 2),1);
decoded(ii, :) = messages(idx, :); % 把预置好的解码矩阵的第ii行写入解
码好的4bit 码字
end

infoBits = reshape(decoded.', 1, []);
% 把解码好的n*4 矩阵重新拼成比特流

if padBits > 0 % 删除补零位
    infoBits = infoBits(1:end-padBits);
end

stats.correctedCount = correctedCount;
stats.totalCodewords = size(codewords, 1);
end
```